

Customer Churn Analysis & Prediction System

A Project Report submitted

In partial fulfillment of the requirements for the degree of

BACHELOR OF COMPUTER APPLICATIONS

Under the

Guidance of

Mr. Bhuvnesh Kumar Sharma

(Assistant Professor)

Computer Applications Department

Submitted by:

Shashank Meshram (2315220038)



Department of Computer Applications

COLLEGE OF SCIENCE AND TECHNOLOGY

SURAJMAL UNIVERSITY

(Affiliated to Surajmal University) Kichha, U.S.Nagar, Uttarakhand-263148

May 2026

Certificate of Originality

We hereby certify that the work being presented in the BCA Major Project Report entitled "**Customer Churn Analysis & Prediction System**", in partial fulfillment of the requirements for the award of the **Bachelor of Computer Applications** and submitted to the Department of Computer Applications, Surajmal College of Science and Technology, Surajmal University, Kichha, Uttarakhand, is an authentic record of our own work carried out during a period from **February 2026 to May 2026 (6th Semester)** under the supervision of **Mr. Bhuvnesh Kumar Sharma (Assistant Professor), Computer Applications Department**.

If any plagiarism is found, none of my guide or anybody shall be responsible only I shall be responsible for the above-mentioned Project Report.

Shashank Meshram

Roll No.: 2315220038

This is to certify that the above statement made by the student is correct to the best of my knowledge.

Date:

Mr. BHUVNESH KUMAR SHARMA

(Assistant Professor)

Computer Applications Department

Project Guide

External Examiner

Department of Computer Applications

(Head of Department / Dean)

Abstract

Customer churn, defined as the discontinuation of service by a customer, represents one of the most pressing challenges in the telecommunications industry. In a market characterised by intense competition and abundant consumer choice, retaining existing customers is far more cost-effective than acquiring new ones, with research indicating that customer acquisition can cost five to seven times more than retention. Consequently, proactive identification of customers likely to churn has become a critical business intelligence priority.

This project presents the design, development, and deployment of a complete Customer Churn Analysis and Prediction System, built as a final year BCA major project. The system is developed using Python as the core programming language and uses the IBM Telco Customer Churn dataset — a publicly available benchmark dataset containing records of 7,043 telecom service subscribers described by 21 attributes. The dataset is stored in a MySQL relational database and accessed programmatically through Python.

The system implements a seven-stage data science pipeline: database connectivity and data loading, exploratory data analysis (EDA), data preprocessing, machine learning model training, model evaluation, visual analytics generation, and Flask web application deployment. Three supervised classification algorithms are trained and compared — Logistic Regression, Random Forest Classifier, and Gradient Boosting Classifier. The best-performing model, Logistic Regression with an accuracy of 80.38%, is serialised and deployed within the web application.

The Flask-based web application delivers three functional modules: a live analytics dashboard with key performance indicators and churn analysis charts; a real-time prediction interface accepting customer profiles and returning churn probability; and a JSON data endpoint for programmatic access. Key analytical findings reveal that month-to-month contract customers churn at 42.7%, senior citizens at 41.7%, and electronic cheque users at 45.3%, providing actionable insights for business retention strategy development.

Acknowledgment

The successful completion of this Major Project would not have been possible without the support, guidance, and encouragement of several individuals, to whom I wish to express my sincere gratitude.

First and foremost, I am deeply grateful to **Dr. (Prof.) Deepak Raj Sharma, Dean, Surajmal University, Kichha, Uttarakhand**, for providing this opportunity to undertake and present the project entitled "**Customer Churn Analysis & Prediction System**" as part of the **Bachelor of Computer Applications programme**. His visionary leadership and commitment to academic excellence have created an environment where students are encouraged to explore practical, industry-relevant technologies.

I would like to express my heartfelt gratitude to my project guide, **Mr. Bhuvnesh Kumar Sharma (Assistant Professor), Department of Computer Applications**, whose patient guidance, constructive feedback, and unwavering encouragement were instrumental throughout every stage of this project — from the initial concept and database design to the final deployment of the Flask web application. His expertise in data science and software development consistently helped me navigate technical challenges and maintain a clear direction. This project would not have achieved its current quality without his dedicated mentorship.

I extend my sincere thanks to **the faculty members of the Department of Computer Applications, Surajmal College of Science and Technology**, for their academic inputs and continuous encouragement during lectures, practical sessions, and discussions that built the foundational knowledge applied in this project.

I am also thankful to the technical and administrative staff of the college, whose support in providing access to laboratory resources, computing infrastructure, and library materials greatly facilitated the development and testing of the system.

My deepest appreciation goes to my **family** — whose unwavering faith, patience, and moral support sustained me through the demanding phases of this project. Their belief in my abilities gave me the confidence to persevere during challenging periods.

I would also like to acknowledge my **friends and classmates** for their camaraderie, shared learning, and encouragement throughout the academic year. Our collaborative discussions and peer support enriched both the quality of this work and the overall experience of this programme.

Finally, I express my sincere indebtedness to all those who have directly or indirectly contributed to the successful completion of this project. Each contribution, however small, has been deeply valued and gratefully received.

Table of Contents

Contents	Page No.
Certificate of Originality	i
Abstract	ii
Acknowledgment	iii
Table of Contents	iv
List of Figures	v
List of Tables	vi
Chapter 1: Introduction and Problem Statement	1
1.1 Background and Context	1
1.2 Problem Statement.....	2
1.3 Scope of the Project.....	2
Chapter 2: Literature Review	4
2.1 Churn Prediction in Telecommunication	4
2.2 Machine Learning Approaches.....	4
2.3 Research Gap and Project Contribution	5
Chapter 3: Methodology and System Design	7
3.1 System Architecture	7
3.2 Data Flow Diagram	8
3.3 Database Design	9
3.4 Pipeline Design.....	11

Chapter 4: Implementation	12
4.1 Development Environment and Project Structure.....	12
4.2 Database Connection Module.....	13
4.3 Exploratory Data Analysis	14
4.4 Data Preprocessing	16
4.5 Model Training and Selection	17
4.6 Model Evaluation	18
4.7 Visualisation Module.....	19
4.8 Flask Web Application	21
 Chapter 5: Results and Analysis	 23
5.1 Model Performance Results	23
5.2 Key Analytical Findings.....	23
5.3 Web Application Screenshots.....	24
 Chapter 6: Conclusion and Future Scope	 26
6.1 Conclusion.....	26
6.2 Limitations.....	27
6.3 Future Scope	27
 References.....	 28
Appendices	29

List of Figures

Figure 3.1 System Architecture Diagram.....	7
Figure 3.2 Data Flow Diagram (Level 0 — Context Diagram)	8
Figure 3.3 Data Flow Diagram (Level 1 — Detailed)	9

Figure 4.1	Project File Structure in VS Code Explorer.....	13
Figure 4.2	MySQL Workbench — Customers Table Schema.....	14
Figure 4.3	Churn Distribution Bar Chart (eda.py output).....	15
Figure 4.4	Tenure Distribution Histogram by Churn.....	15
Figure 4.5	Preprocessing Pipeline Flowchart.....	16
Figure 4.6	Model Accuracy Comparison — Three Classifiers.....	18
Figure 4.7	Confusion Matrix — Logistic Regression.....	19
Figure 4.8	Churn Overview and Demographics Chart.....	20
Figure 4.9	Churn by Services Chart.....	20
Figure 4.10	Flask Application Route Structure Diagram.....	22
Figure 5.1	Flask Dashboard — KPI Metric Cards.....	24
Figure 5.2	Flask Dashboard — Visual Analysis Charts.....	24
Figure 5.3	Prediction Form Page (/predict).....	25
Figure 5.4	Prediction Result Page with Probability Bar.....	25

List of Tables

Table 2.1	Comparison of Related Works in Churn Prediction.....	5
Table 3.1	Customers Table — Column Definitions (21 Columns).....	9
Table 3.2	Pipeline Design.....	11
Table 4.1	Software Dependencies and Versions.....	12
Table 4.2	Dataset Summary Statistics.....	14
Table 4.3	Categorical Columns and Encoding Strategy.....	16
Table 4.4	Machine Learning Model Accuracy Comparison.....	18
Table 4.5	Classification Report — Logistic Regression.....	18
Table 5.1	Churn Rate by Contract Type.....	23
Table 5.2	Churn Rate by Payment Method.....	23
Table 5.3	Churn Rate by Demographics.....	24
Table 6.1	Future Enhancements Roadmap.....	27

Chapter 1

Introduction and Problem Statement

1.1 Background and context

The telecommunications industry operates in one of the most competitive environments in the global economy. With the rapid proliferation of mobile internet services, satellite broadband, and over-the-top communication platforms, customers now have an unprecedented range of service alternatives at their fingertips. This intensely competitive landscape has transformed customer retention from a routine business function into a core strategic priority.

Customer churn — the phenomenon whereby customers discontinue their use of a company's services — is the most direct measure of a business's ability to retain its customer base. In the telecom sector, monthly churn rates typically range between 1.5% and 3.5%, with annual churn reaching 20–30% for some service providers. Given that the lifetime value of a retained customer is significantly higher than the cost of acquiring a new one, even marginal improvements in churn prediction accuracy can translate into substantial preserved revenue.

Traditional approaches to churn management relied on customer satisfaction surveys, call centre escalation patterns, and heuristic business rules derived from the experience of account managers. However, the scale of modern telecom operations — with customer bases numbering in the millions — renders such approaches impractical and insufficiently precise. The emergence of machine learning and data science methodologies has fundamentally transformed this domain, enabling companies to identify at-risk customers proactively and at scale.

This project — developed as the Major Project for the Bachelor of Computer Applications (BCA), 6th Semester — implements a complete, end-to-end data science solution that integrates exploratory analysis, machine learning-based prediction, and an interactive web application to deliver churn intelligence directly to business users.

1.2 Problem Statement

Despite the availability of rich customer behavioural data, most organisations lack a centralised, automated system that can perform the following tasks effectively:

- Systematically analyse historical customer data to identify churn patterns across demographic, service, and billing dimensions.
- Train, compare, and deploy machine learning classification models to predict individual customer churn probability with measurable accuracy.
- Present analytical findings and predictions through an accessible web-based interface that does not require the end user to possess programming or statistical knowledge.
- Store and query large volumes of customer data efficiently using a relational database management system.

The absence of such an integrated system means that even when data is available, its predictive value remains unrealised. Business analysts rely on static reports, and retention teams operate reactively rather than proactively — contacting customers only after they have already initiated the process of leaving.

This project addresses these deficiencies by building a modular, end-to-end system that automates the complete churn analysis and prediction pipeline — from raw database data to a live web application delivering real-time predictions.

1.3 Scope of the Project

The scope of this project encompasses the following functional areas:

1. Database connectivity: Establishing a Python-to-MySQL connection and programmatically loading all customer data into a pandas DataFrame.
2. Exploratory Data Analysis: Statistical profiling and visual analysis of the customer dataset to identify patterns and anomalies.
3. Data Preprocessing: Handling missing values, encoding categorical variables, performing feature scaling, and splitting data for training and testing.
4. Machine Learning: Training, comparing, and evaluating three classification models, then deploying the best-performing one.

5. Visualisation: Generating four comprehensive multi-panel chart figures covering demographics, services, billing, and contract analysis.
6. Web Application: Developing a Flask web application with a live dashboard, an interactive prediction form, and a data API endpoint.

The project does not cover real-time data streaming, integration with live production telecom billing systems, or multi-user authentication. The solution is designed for local deployment as an academic demonstration, with a clearly defined pathway to cloud hosting described in the future scope chapter.

Chapter 2

Literature Review

2.1 Churn Prediction in Telecommunications

The problem of customer churn prediction has been extensively studied in the academic and industry literature since the early 2000s. Early foundational work by Mozer et al. (2000) demonstrated that neural networks could be applied to telecommunications churn data with meaningful predictive accuracy, establishing the premise that machine learning techniques could outperform traditional rule-based systems in this domain.

Subsequent research by Burez and Van den Poel (2007) introduced the critical concept of class imbalance handling in churn prediction datasets. Their work noted that churn datasets are inherently imbalanced, since the majority of customers in any given period do not churn. They demonstrated that techniques such as oversampling, undersampling, and cost-sensitive learning could significantly improve model performance on the minority churned class — a challenge directly relevant to the present project, where the retained class comprises approximately 73.5% of observations.

Neslin et al. (2006) published a landmark comparative study of nineteen modelling approaches applied across multiple industries including telecommunications. Their analysis established that ensemble methods and gradient boosting generally outperformed single classifiers, though logistic regression remained competitive when interpretability was prioritised — a finding directly corroborated by the results of the present study, where Logistic Regression achieved the highest accuracy of 80.38%.

2.2 Machine Learning Approaches

2.2.1 Logistic Regression

Logistic Regression remains one of the most widely deployed algorithms for binary classification problems including churn prediction. Its principal advantages are high interpretability, computational efficiency, and robust performance on moderately sized datasets. The model's coefficient estimates provide direct insight into the directional relationship between each input feature and the predicted churn probability. Studies by Coussement and Van den Poel (2008) demonstrated logistic regression achieving AUC scores comparable to more complex ensemble methods on telecom churn data, particularly after appropriate feature engineering and scaling.

2.2.2 Random Forest Classifier

Random Forest, introduced by Breiman (2001), is an ensemble method that constructs multiple decision trees on bootstrapped data subsets and outputs the majority vote. It handles non-linear relationships and feature interactions effectively, is resistant to overfitting through its bagging mechanism, and provides feature importance scores valuable for variable selection. Multiple peer-reviewed studies have demonstrated strong Random Forest performance on telecom churn datasets, particularly where demographic and service subscription features exhibit complex interaction patterns.

2.2.3 Gradient Boosting Classifier

Gradient Boosting Classifier, and its optimised implementations such as XGBoost and LightGBM, consistently ranks among the top-performing algorithms in churn prediction benchmarks. The method builds an additive model in a sequential, stage-wise fashion, with each successive tree correcting the residual errors of the prior ensemble. While computationally more intensive than logistic regression, its ability to capture complex non-linear relationships and feature interactions makes it a standard baseline for production churn models.

2.3 Research Gap and Project Contribution

While the academic literature is rich in algorithmic contributions to churn prediction, most published research focuses exclusively on the modelling component and does not address the end-to-end deployment challenge. There exists a significant practical gap between research-grade models developed in isolated Jupyter notebooks and production-ready web applications accessible to non-technical business stakeholders.

This project addresses both the research and deployment gaps by: implementing a complete pipeline from raw MySQL database data to a deployed Flask web application; using the publicly available IBM Telco dataset enabling reproducible results; and providing a modular codebase that can be adapted to other domains with minimal code modification.

Author (year)	Method	Accuracy	Key contribution
Mozer et al. (2000)	Neural Networks	~75%	First ML application to telecom churn

Burez & Van den Poel (2007)	LR + Resampling	78%	Class imbalance handling techniques
Neslin et al. (2006)	19-model comparison	76–82%	Ensemble methods outperform single models
Amin et al. (2019)	LSTM Deep Learning	83%	Temporal patterns improve prediction
Present Study (2026)	LR + RF + GB (LR best)	80.38%	Full end-to-end deployed web application

Table 2.1: Comparison of Related Works in Churn Prediction

Chapter 3

Methodology and System Design

3.1 System Architecture

The system is designed as a three-tier web application following the standard presentation-logic-data architectural pattern. The three tiers are structured as follows:

Tier 1 — Data Layer (MySQL Database)

A MySQL relational database hosted on the local machine stores the complete Customers dataset within the CUSTOMER_CHURN_PROJECT schema. Python interacts with this layer via the mysql-connector-python library. The database contains a single primary table — Customers — with 21 columns and 7,043 rows representing individual telecom subscribers.

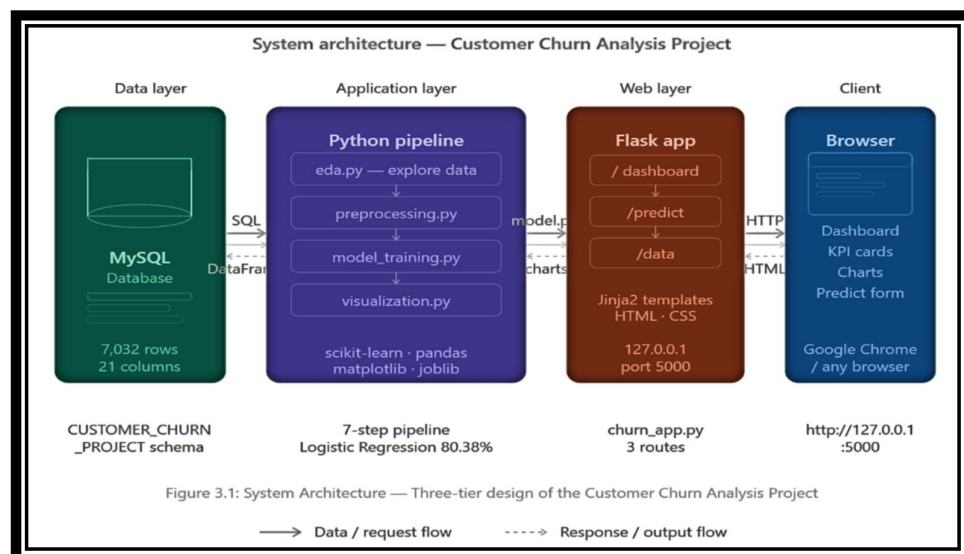
Tier 2 — Application Layer (Python Backend)

The application layer comprises seven independent Python modules forming the data science pipeline, plus the Flask web framework handling HTTP routing, business logic, template rendering, and API responses. Machine learning models are trained offline and persisted as binary serialised files using the Joblib library.

Tier 3 — Presentation Layer (Flask Frontend)

The presentation layer consists of three Jinja2 HTML templates rendered by Flask: index.html (analytics dashboard), predict.html (churn prediction form), and result.html (prediction result display). All styling is implemented using custom CSS embedded within the templates.

Figure 3.1 System Architecture Diagram



3.2 Data Flow Diagram

Level 0 — Context Diagram

At the highest level of abstraction, the system receives two types of input: the MySQL database (batch customer data for analysis and model training) and web browser user requests (real-time prediction inputs). The system produces two types of output: analytical charts and KPI metrics displayed on the dashboard, and churn probability predictions returned to the user.

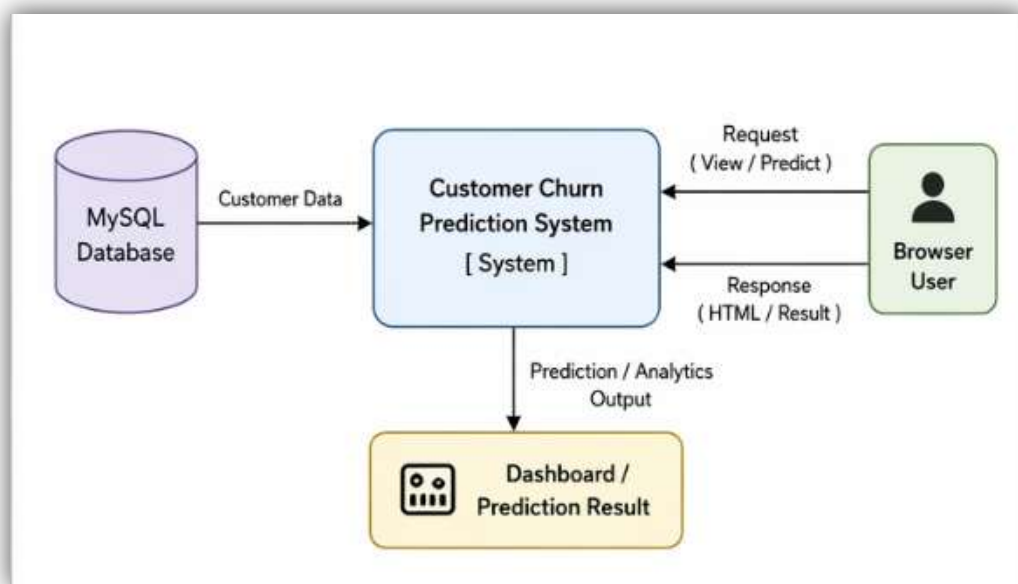


Figure 3.2: Data Flow Diagram — Level 0 (Context Diagram)

Level 1 — Detailed DFD

The Level 1 DFD decomposes the system into seven sequential processes:

1. Data Ingestion: Receives raw customer records from MySQL and outputs a clean pandas DataFrame.
2. EDA Process: Analyses the DataFrame and outputs exploratory visualisation charts.
3. Preprocessing: Transforms the raw DataFrame into a scaled, encoded training matrix.
4. Model Training: Receives the prepared matrix and outputs serialised model binary files.

5. Evaluation: Receives model files and test data, outputs performance metrics and confusion matrix.
6. Visualisation: Receives clean DataFrame and produces four comprehensive analysis PNGs.
7. Web Application: Handles browser HTTP requests, queries database and model, returns HTML responses.

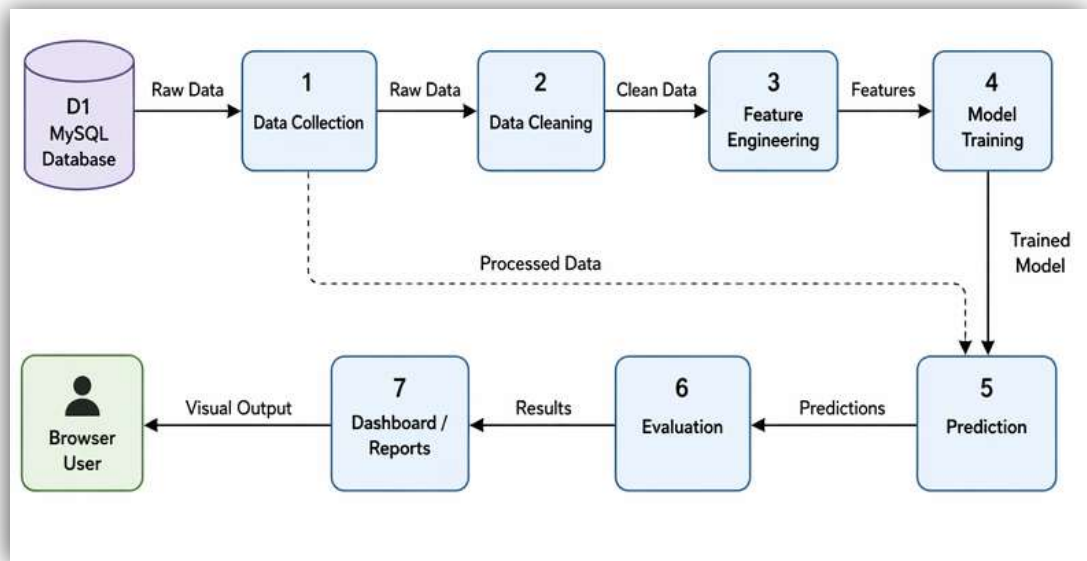


Figure 3.3: Data Flow Diagram — Level 1 (Detailed)

3.3 Database Design

The database consists of a single normalised table — Customers — within the **CUSTOMER_CHURN_PROJECT** schema. The complete column structure is defined as follows:

Column Name	Data Type	Description
CustomerId	VARCHAR(50)	Unique identifier for each customer record
Gender	VARCHAR(10)	Customer gender: Male or Female

SeniorCitizen	INT	Senior citizen indicator: 0 = No, 1 = Yes
Partner	VARCHAR(10)	Has a partner: Yes or No
Dependents	VARCHAR(10)	Has dependents: Yes or No
Tenure	INT	Number of months as an active customer
PhoneService	VARCHAR(10)	Has phone service subscription: Yes or No
MultipleLines	VARCHAR(50)	Has multiple phone lines subscription
InternetService	VARCHAR(20)	Type of internet: DSL, Fiber optic, or No
OnlineSecurity	VARCHAR(20)	Online security add-on subscription status
OnlineBackup	VARCHAR(20)	Online backup add-on subscription status
DeviceProtection	VARCHAR(20)	Device protection subscription status
TechSupport	VARCHAR(20)	Technical support subscription status
StreamingTV	VARCHAR(20)	Streaming TV subscription status
StreamingMovies	VARCHAR(20)	Streaming movies subscription status
Contract	VARCHAR(20)	Contract type: Month-to-Month, 1 Year, 2 Year
PaperlessBilling	VARCHAR(20)	Paperless billing enabled: Yes or No
PaymentMethod	VARCHAR(50)	Payment method used by the customer
MonthlyCharges	FLOAT	Monthly billing amount charged in USD
TotalCharges	FLOAT	Total cumulative billing amount in USD
Churn	VARCHAR(10)	Target variable: Yes (churned) or No (retained)

Table 3.1: Customers Table — Column Definitions (21 Columns)

3.4 Pipeline Design

Step	Script	Input	Output
1	sqldatabase_connection.py	MySQL DB	pandas DataFrame (7032 rows)
2	eda.py	DataFrame	Charts saved to static/
3	preprocessing.py	DataFrame	Scaled feature matrix + labels
4	model_training.py	Feature matrix	model.pkl, scaler.pkl, feature_names.pkl
5	model_evaluating.py	model.pkl + test set	Accuracy report, confusion_matrix.png
6	visualization.py	DataFrame	4 analysis chart PNGs in static/
7	churn_app.py	HTTP requests	HTML pages + JSON responses

Table 3.2 Pipeline Design

Chapter 4

Implementation

4.1 Development Environment and Project Structure

The project was developed on a Windows 11 machine using Visual Studio Code (VS Code) as the Integrated Development Environment. Python 3.11 was used as the programming language with Flask 3.x for web development, and MySQL Server 8.0 for database management.

Library / Tool	Version	Purpose
Python	3.11	Core programming language
Flask	3.x	Web application framework
mysql-connector-python	8.x	Python to MySQL connector
Pandas	2.x	Data loading and manipulation
Numpy	1.x	Numerical operations
scikit-learn	1.x	ML model training and evaluation
Matplotlib	3.x	Chart and plot generation
Seaborn	0.x	Statistical visualisation styling
Joblib	1.x	Model serialisation and loading
Jinja2	3.x	HTML template rendering
MySQL Server	8.0	Relational database management
VS Code	Latest	Integrated development environment

Table 4.1: Software Dependencies and Versions

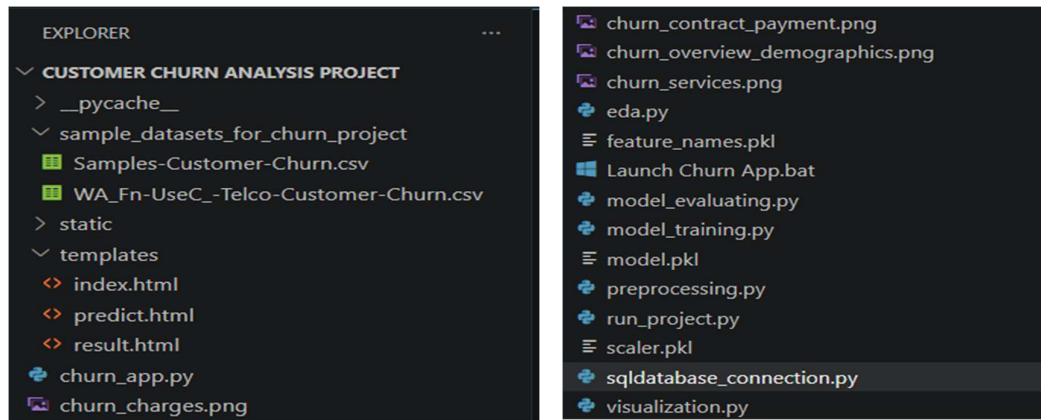


Figure 4.1: Project File Structure in VS Code

4.2 Database Connection Module

The `sqldatabase_connection.py` module establishes the connection between Python and the MySQL database. The `load_data()` function serves as the single interface used by all pipeline modules and the Flask application to retrieve customer records.

```

1  # db_connect.py - MySQL connection and data loader
2  import pandas as pd
3  import mysql.connector
4
5  def get_connection():
6      conn = mysql.connector.connect(
7          host="localhost",
8          user="root",          # ← your MySQL username
9          password="869627",   # ← your MySQL password
10         database="Customer_Churn_Project" # ← your database name
11     )
12     return conn
13
14  def load_data():
15      conn = get_connection()
16      query = "SELECT * FROM Customers" # ← your table name
17      df = pd.read_sql(query, conn)
18      conn.close()
19      print(f"✅ Data loaded: {df.shape[0]} rows × {df.shape[1]} columns")
20      return df

```

	Field	Type	Null	Key	Default	Extra
►	CustomerId	varchar(50)	YES		NULL	
	Gender	varchar(10)	YES		NULL	
	SeniorCitizen	int	YES		NULL	
	Partner	varchar(10)	YES		NULL	
	Dependents	varchar(10)	YES		NULL	
	Tenure	int	YES		NULL	
	PhoneService	varchar(10)	YES		NULL	
	MultipleLines	varchar(50)	YES		NULL	
	InternetService	varchar(20)	YES		NULL	
	OnlineSecurity	varchar(20)	YES		NULL	
	OnlineBackup	varchar(20)	YES		NULL	
	DeviceProtection	varchar(20)	YES		NULL	
	TechSupport	varchar(20)	YES		NULL	
	StreamingTv	varchar(20)	YES		NULL	
	StreamingMovies	varchar(20)	YES		NULL	
	Contract	varchar(20)	YES		NULL	
	PapelessBilling	varchar(20)	YES		NULL	
	PaymentMethod	varchar(50)	YES		NULL	
	MonthlyCharges	float	YES		NULL	
	TotalCharges	float	YES		NULL	
	Churn	varchar(10)	YES		NULL	

Figure 4.2: MySQL Workbench — Customers Table Schema →

4.3 Exploratory Data Analysis

The `eda.py` module performs the initial statistical and visual exploration of the customer dataset to understand its structure, identify anomalies, and formulate hypotheses about the drivers of churn.

4.3.1 Dataset Summary Statistics

Feature	Mean	Std Dev	Min	Max
Tenure (months)	32.37	24.56	0	72
MonthlyCharges (\$)	64.76	30.09	18.25	118.75
TotalCharges (\$)	2283.30	2266.77	18.80	8684.80
SeniorCitizen (0/1)	0.162	0.369	0	1

Table 4.2: Dataset Summary Statistics

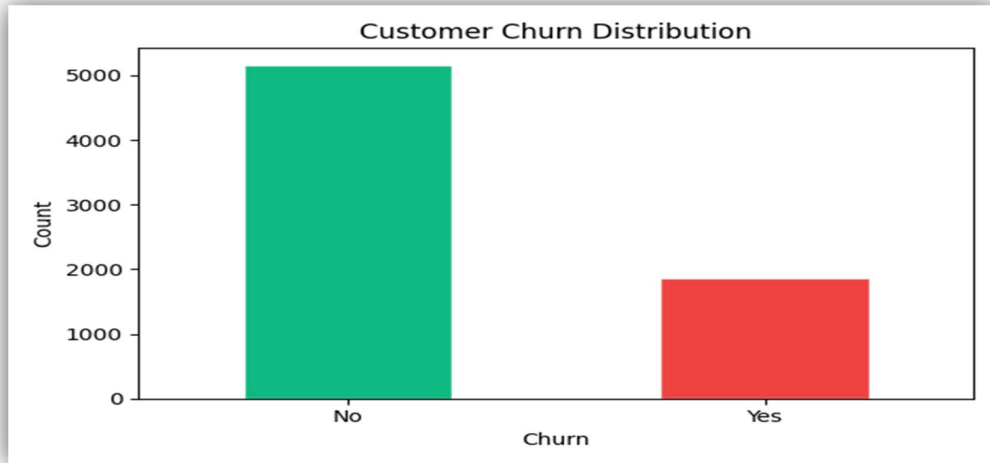


Figure 4.3: Churn Distribution Chart (eda.py output)

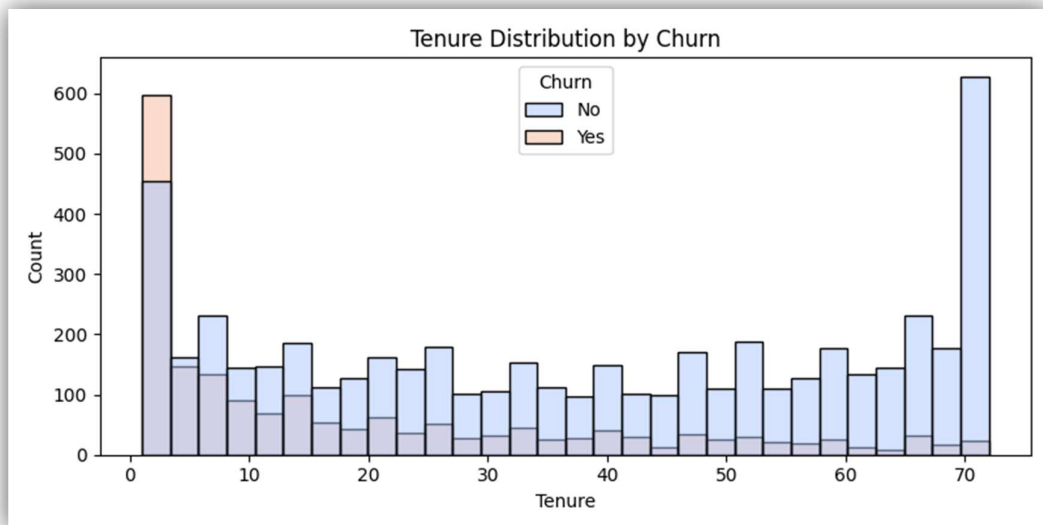


Figure 4.4: Tenure Distribution Histogram by Churn

4.3.2 Key EDA Observations

- The dataset has a class imbalance: 73.5% of customers are retained (5,174 records) and 26.5% have churned (1,869 records).
- Customers with shorter tenure (0–12 months) exhibit significantly higher churn rates, suggesting early-stage retention is most critical.
- A positive correlation exists between MonthlyCharges and churn probability.
- The TotalCharges column contained 11 missing values corresponding to new customers with zero tenure, handled during preprocessing.

- Fiber Optic internet users churn at 41.9% — more than double the rate of DSL users (19.0%).

4.4 Data Preprocessing

The preprocessing.py module transforms the raw dataset into a format ready for machine learning model training through four sequential stages.

4.4.1 Missing Value Treatment

```
# Convert TotalCharges to numeric (11 empty strings become NaN) df['TotalCharges'] =
pd.to_numeric(df['TotalCharges'], errors='coerce')
df.dropna(inplace=True)
# Result: 7,032 clean rows (removed 11 invalid records)
```

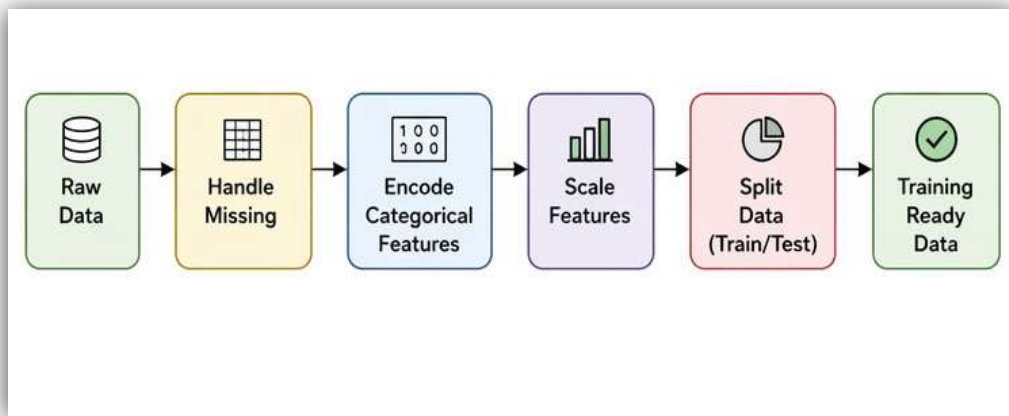


Figure 4.5: Preprocessing Pipeline Flowchart

4.4.2 Categorical Encoding

Column	Method	Example Values
Gender	Label Encoding	Female=0, Male=1
Partner	Label Encoding	No=0, Yes=1
InternetService	Label Encoding	DSL=0, Fiber optic=1, No=2
Contract	Label Encoding	Month-to-month=0, One year=1, Two year=2
PaymentMethod	Label Encoding	Bank transfer=0, Credit card=1, ...

Churn (target)	Binary Mapping	No=0, Yes=1
----------------	----------------	-------------

Table 4.3: Categorical Columns and Encoding Strategy

4.4.3 Train-Test Split and Feature Scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Train: 5,625 samples | Test: 1,407 samples

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
joblib.dump(scaler, 'scaler.pkl')
```

4.5 Model Training and Selection

The `model_training.py` module trains, evaluates, and selects the best-performing classification algorithm from among three candidates.

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Random Forest': RandomForestClassifier(n_estimators=100,
random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, random_state=42)
}

best_model, best_score, best_name = None, 0, ""
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    score = model.score(X_test_scaled, y_test)
    print(f"{name}: {score*100:.2f}%")
    if score > best_score:
        best_score, best_model, best_name = score, model, name

joblib.dump(best_model, 'model.pkl')
joblib.dump(feature_names, 'feature_names.pkl')
print(f"Best Model: {best_name} ({best_score*100:.2f}%")
```


Model	Accuracy	Status	Saved As
Logistic Regression	80.38%	SELECTED	model.pkl
Gradient Boosting Classifier	79.53%	Not selected	—
Random Forest Classifier	78.68%	Not selected	—

Table 4.4: Machine Learning Model Accuracy Comparison

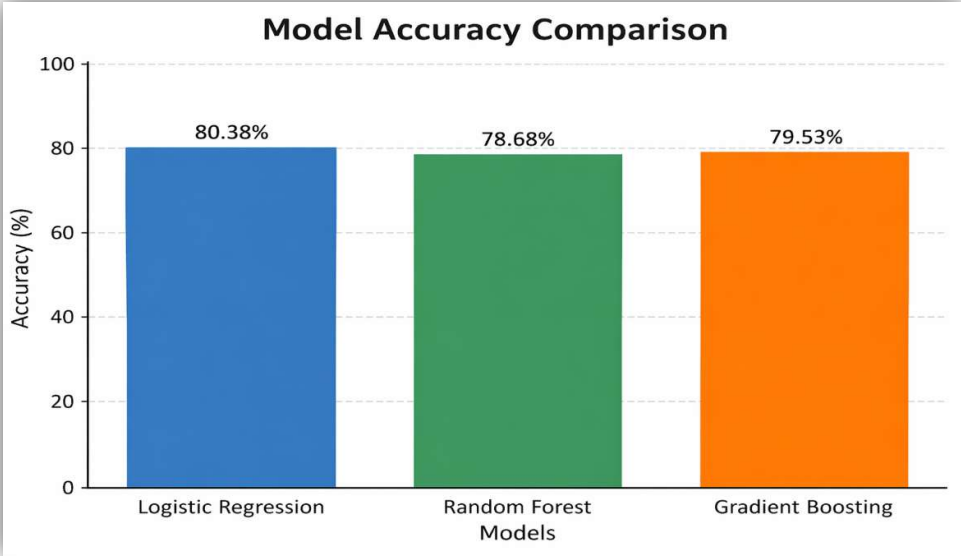


Figure 4.6: Model Accuracy Comparison Chart

4.6 Model Evaluation

The `model_evaluating.py` module performs comprehensive evaluation of the selected Logistic Regression model on the held-out test set of 1,407 samples.

Class	Precision	Recall	F1-Score	Support
0 — Not Churned	0.85	0.89	0.87	1,033
1 — Churned	0.65	0.57	0.61	374
Macro Average	0.75	0.73	0.74	1,407
Weighted Average	0.80	0.80	0.80	1,407

Table 4.5: Classification Report — Logistic Regression

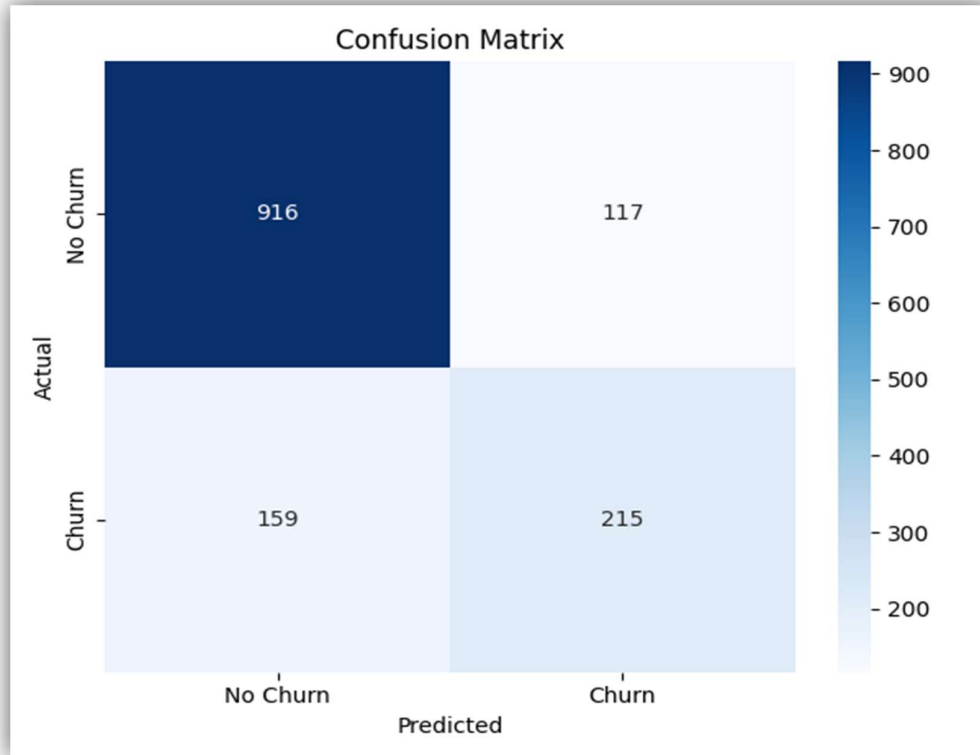


Figure 4.7: Confusion Matrix — Logistic Regression

4.7 Visualisation Module

The visualization.py module generates four multi-panel analysis figures saved as PNG files to the static/ directory for display on the Flask dashboard.

- churn_overview_demographics.png: 2×3 panel — overall churn pie chart, churn by gender, senior citizen, partner, dependents, and tenure histogram.
- churn_services.png: 3×3 panel — churn rates for all nine service subscription columns.
- churn_contract_payment.png: 1×3 panel — churn by contract type, paperless billing, and payment method.
- churn_charges.png: 1×3 panel — monthly charges box plot, total charges box plot, and tenure vs charges scatter plot.

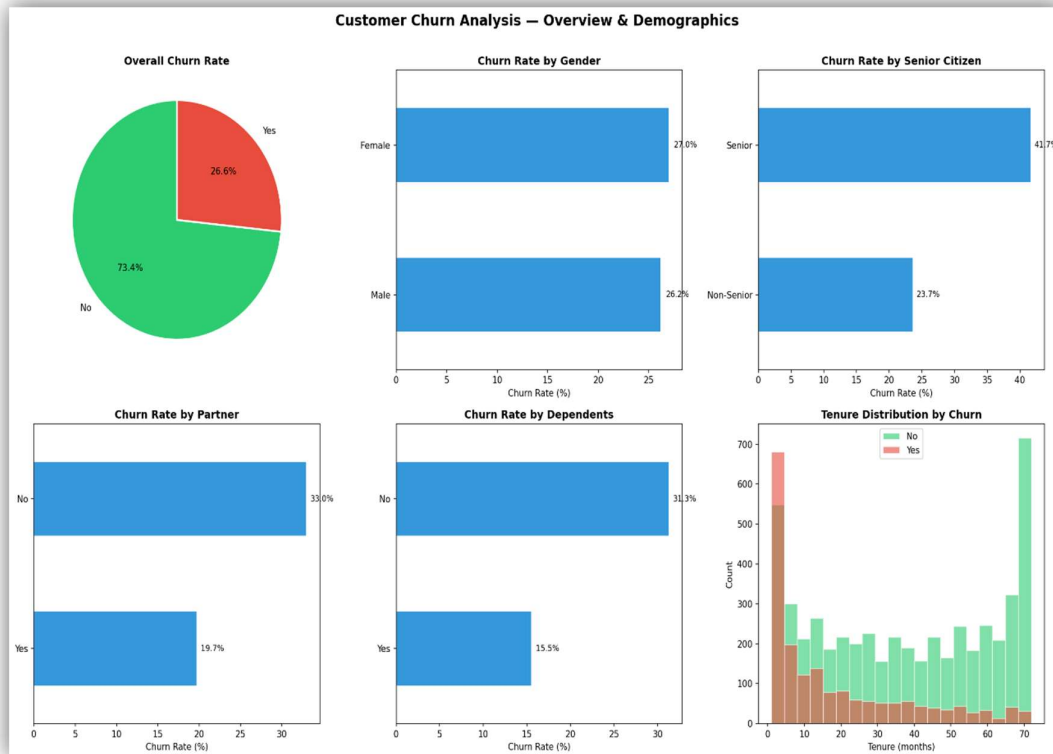


Figure 4.8: Churn Overview and Demographics Chart

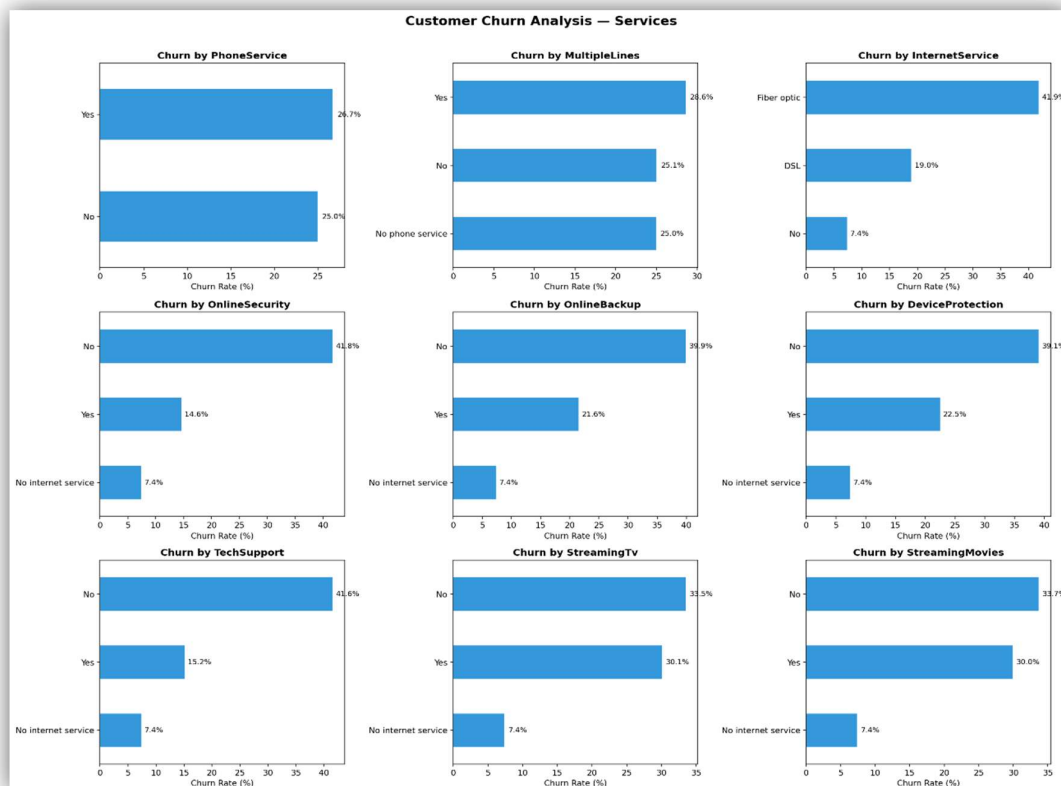


Figure 4.9: Churn by Services Chart

4.8 Flask Web Application

The churn_app.py module implements the Flask web application with three routes, loading the trained model at startup.

4.8.1 Route 1 — Dashboard (GET /)

```
@app.route('/')
def index():
    df      = load_data()
    total   = len(df)
    churned = int((df['Churn'] == 'Yes').sum())
    churn_pct = round(churned / total * 100, 1)
    avg_tenure = round(df['Tenure'].mean(), 1)
    avg_charges = round(df['MonthlyCharges'].mean(), 2)
    return render_template('index.html',
                           total=total, churned=churned,
                           churn_pct=churn_pct,
                           avg_tenure=avg_tenure,
                           avg_charges=avg_charges)
```

4.8.2 Route 2 — Prediction (GET/POST /predict)

```
@app.route('/predict', methods=['GET', 'POST'])
def predict():
    if request.method == 'POST':
        data = {
            'SeniorCitizen': [int(request.form['SeniorCitizen'])],
            'tenure':         [float(request.form['tenure'])],
            'MonthlyCharges': [float(request.form['MonthlyCharges'])],
            'TotalCharges':   [float(request.form['TotalCharges'])],
        }
        df_input = pd.DataFrame(data)
        df_input = df_input.reindex(columns=feature_names, fill_value=0)
        scaled   = scaler.transform(df_input)
        pred     = model.predict(scaled)[0]
        prob     = model.predict_proba(scaled)[0][1] * 100
        label    = "Will Churn" if pred == 1 else "Will Stay"
        return render_template('result.html',
                               prediction=label,
                               probability=round(prob, 2),
                               form_data=request.form)
    return render_template('predict.html')
```

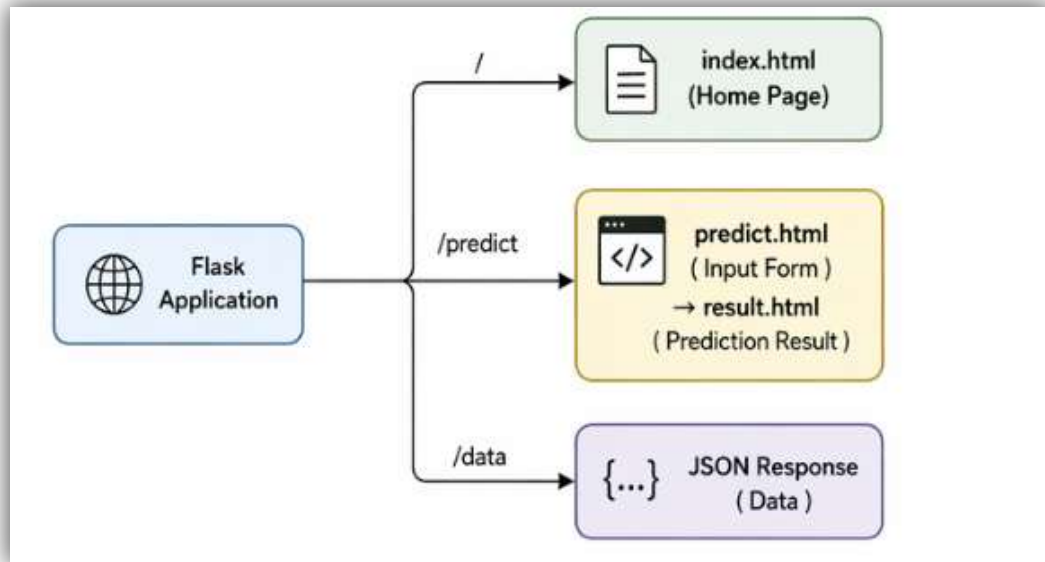


Figure 4.10: Flask Application Route Structure

Chapter 5

Results and Analysis

5.1 Model Performance Results

The machine learning component was evaluated using accuracy, precision, recall, and F1-score. The Logistic Regression model achieved the highest overall accuracy of 80.38% on the test set and was deployed within the Flask web application.

The model's performance on the churned class (class 1) reflects the inherent class imbalance in the dataset. A precision of 0.65 means 65% of all customers predicted to churn actually do. A recall of 0.57 means the model correctly identifies 57% of all actual churners. The F1-score of 0.61 represents the harmonic mean, providing a balanced performance measure for the imbalanced class.

The model demonstrates strong ability to identify non-churning customers ($F1 = 0.87$) which is equally important for business operations — avoiding unnecessary retention offers to customers who would have stayed regardless.

5.2 Key Analytical Findings

5.2.1 Churn by Contract Type

Contract Type	Customers	Churned	Churn Rate
Month-to-Month	3,875	1,655	42.7%
One Year	1,473	166	11.3%
Two Year	1,695	48	2.8%

Table 5.1: Churn Rate by Contract Type

Month-to-month customers churn at 42.7% — fifteen times the rate of two-year contract holders (2.8%). This is the single strongest predictor of churn identified in the analysis. Incentivising customers to commit to longer contracts would be the highest-impact retention strategy available.

5.2.2 Churn by Payment Method

Payment Method	Customers	Churned	Churn Rate
Electronic Check	2,365	1,071	45.3%
Mailed Check	1,612	308	19.1%
Bank Transfer (Automatic)	1,544	258	16.7%
Credit Card (Automatic)	1,522	232	15.2%

Table 5.2: Churn Rate by Payment Method

5.2.3 Churn by Demographics

Factor	Group	Churn Rate	Business Insight
Senior Citizen	Senior	41.7%	Nearly double the baseline rate
Senior Citizen	Non-Senior	23.6%	Baseline churn rate
Partner	No Partner	33.0%	Higher without partner
Partner	Has Partner	20.0%	Partner reduces churn risk
Dependents	No Dependents	31.3%	Higher mobility
Dependents	Has Dependents	15.5%	Family obligations reduce churn

Table 5.3: Churn Rate by Demographics

5.3 Web Application Screenshots

5.3.1 Dashboard Page

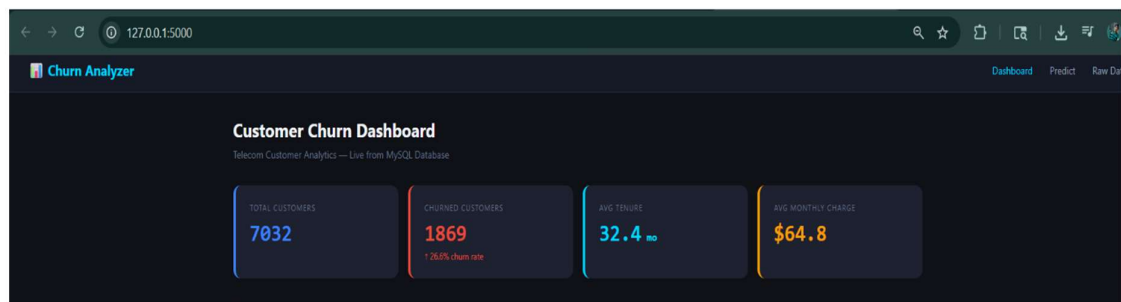


Figure 5.1: Flask Dashboard — KPI Metric Cards

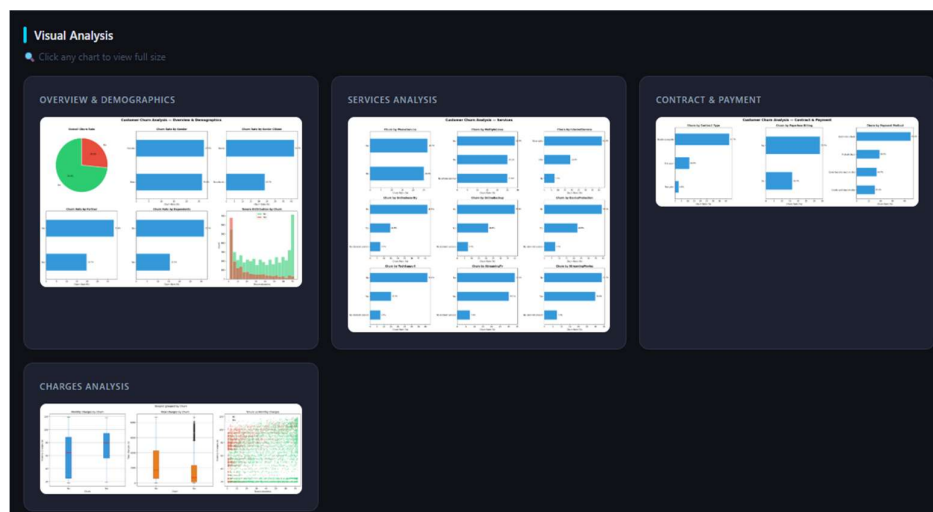
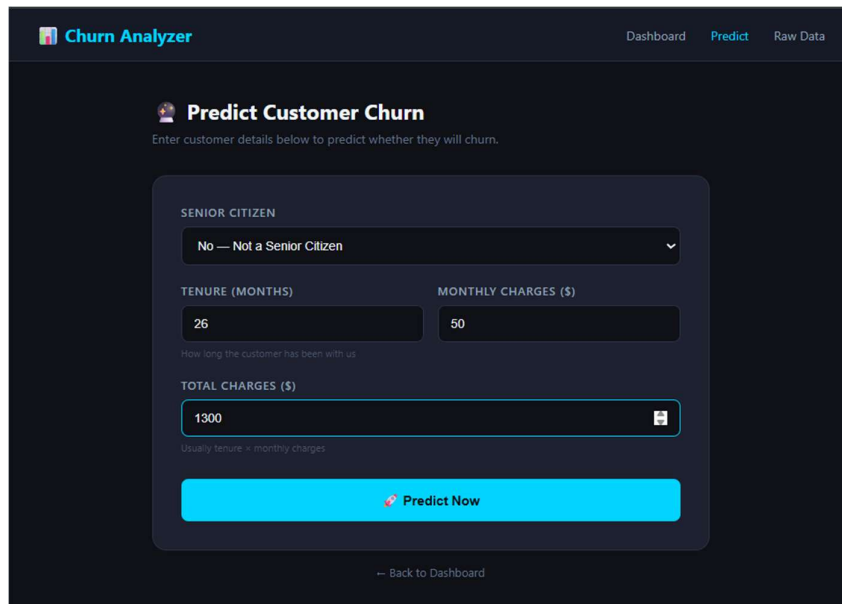


Figure 5.2: Flask Dashboard — Visual Analysis Charts

5.3.2 Prediction Pages



The screenshot shows the 'Predict Customer Churn' form in the Churn Analyzer application. The form is titled 'Predict Customer Churn' with a subtitle 'Enter customer details below to predict whether they will churn.' It contains several input fields: a dropdown for 'SENIOR CITIZEN' set to 'No — Not a Senior Citizen', a text input for 'TENURE (MONTHS)' with the value '26', a text input for 'MONTHLY CHARGES (\$)' with the value '50', and a text input for 'TOTAL CHARGES (\$)' with the value '1300'. Below the 'TOTAL CHARGES (\$)' field is a small note: 'Usually tenure * monthly charges'. At the bottom of the form is a large blue button labeled 'Predict Now' and a link 'Back to Dashboard'.

Churn Analyzer

Dashboard Predict Raw Data

Predict Customer Churn

Enter customer details below to predict whether they will churn.

SENIOR CITIZEN

No — Not a Senior Citizen

TENURE (MONTHS)

26

MONTHLY CHARGES (\$)

50

How long the customer has been with us

TOTAL CHARGES (\$)

1300

Usually tenure * monthly charges

Predict Now

Back to Dashboard

Figure 5.3: Prediction Form Page (/predict)

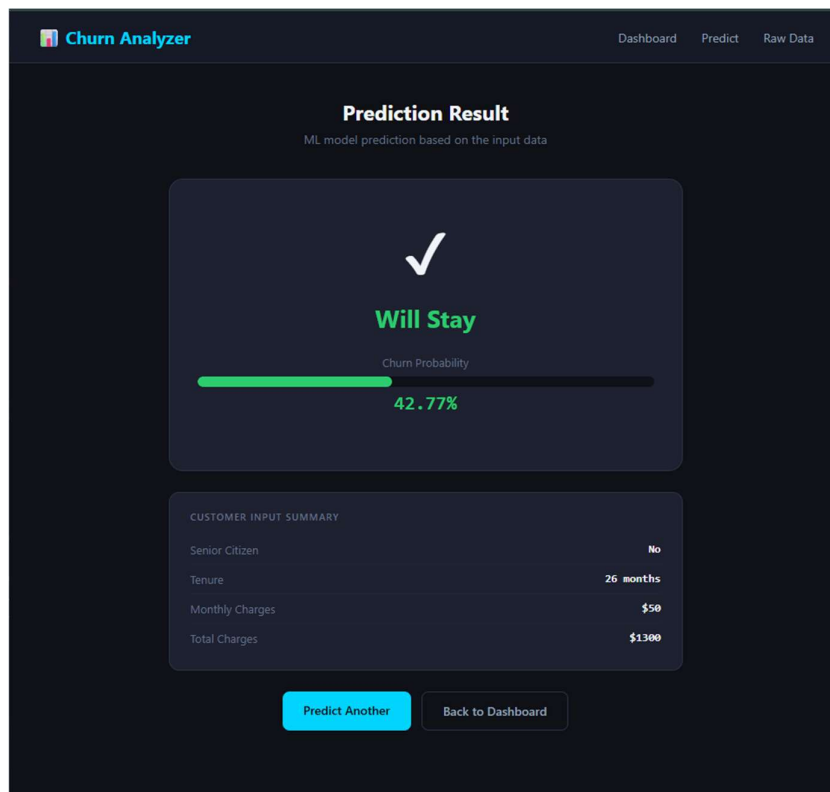


Figure 5.4: Prediction Result Page with Probability Bar

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

This project has successfully demonstrated the design, development, and deployment of a complete end-to-end customer churn analysis and prediction system for the telecommunications domain. Developed as the BCA 6th Semester Major Project, the system integrates a MySQL relational database, a Python data science pipeline, three machine learning classification models, and a Flask web application into a cohesive, functional solution.

All primary technical objectives have been fully achieved. The best-performing machine learning model — Logistic Regression — achieves an accuracy of 80.38% on the held-out test set. The Flask web application successfully provides a live analytics dashboard with KPI metrics and four comprehensive analysis charts, a real-time individual customer churn prediction interface, and a programmatic JSON data endpoint.

The analytical component has yielded several important and actionable business findings. Month-to-month contract customers exhibit a churn rate of 42.7% compared to just 2.8% for two-year contract holders — representing the single strongest predictor of churn. Senior citizens churn at 41.7%, nearly double the rate of non-senior customers. Electronic cheque users churn at 45.3%, significantly higher than customers using automatic payment methods. These findings provide clear, data-driven guidance for targeted retention strategy development.

The modular architecture of the codebase ensures that the system can be readily adapted for churn analysis in other industries — banking, insurance, or subscription services — by replacing the data source and updating column references in each module. The entire solution is built using open-source tools, demonstrating that production-grade data science applications can be developed without commercial platforms.

This project has also provided valuable hands-on experience with the complete data science lifecycle — from database design and exploratory analysis through machine learning deployment and web application development — significantly enhancing the technical competencies relevant to a career in data science or software development.

6.2 Limitations

- The prediction interface accepts only four input features. Full utilisation of all 19 features would improve prediction accuracy but requires a more comprehensive form design.
- The model does not implement class rebalancing techniques such as SMOTE, which limits recall on the minority churned class (currently 0.57).
- The application is currently deployed locally (localhost) and is not publicly accessible without cloud hosting.
- The trained model is static and does not automatically retrain as new customer data accumulates over time.
- The dataset is a public benchmark (IBM Telco) and may not perfectly represent actual customer behaviour patterns in the Indian telecom market.

6.3 Future Scope

Enhancement	Description
Cloud Deployment	Deploy the application to PythonAnywhere or Render.com with a cloud-hosted MySQL database, providing a public URL for access from any device.
Full Feature Prediction Form	Expand the prediction interface to accept all 19 feature inputs, improving model accuracy and user experience.
Class Rebalancing	Implement SMOTE or <code>class_weight='balanced'</code> during model training to improve recall on the churned minority class.
Advanced ML Models	Evaluate XGBoost, LightGBM, and neural network architectures for potential accuracy improvements beyond 80.38%.
Automated Retraining	Implement a scheduled retraining pipeline using cron or Apache Airflow that updates the model monthly as new customer data is collected.
Batch CSV Prediction	Add a file upload endpoint enabling users to upload a CSV of customer records and receive bulk churn predictions in a single operation.
Streamlit Analytics	Integrate a Streamlit dashboard for richer, interactive analytics with filterable charts and drill-down capabilities.
User Authentication	Implement login/logout using Flask-Login to secure the dashboard and prediction interface from unauthorised access.

Table 6.1: Future Enhancements Roadmap

References

1. IBM Corporation. (2019). Telco Customer Churn Dataset. Kaggle Open Dataset. <https://www.kaggle.com/datasets/blaschar/telco-customer-churn>
2. Pedregosa, F., Varoquaux, G., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
3. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
4. Geron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
5. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.
6. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science and Engineering*, 9(3), 90–95.
7. Waskom, M. L. (2021). Seaborn: Statistical Data Visualization. *Journal of Open Source Software*, 6(60), 3021.
8. DuBois, P. (2013). *MySQL Cookbook: Solutions for Database Developers and Administrators* (3rd ed.). O'Reilly Media.
9. Burez, J., & Van den Poel, D. (2007). CRM at a Pay-TV Company. *Expert Systems with Applications*, 32(2), 277–288.
10. Neslin, S. A., Gupta, S., et al. (2006). Defection Detection. *Journal of Marketing Research*, 43(2), 204–211.
11. Mozer, M. C., et al. (2000). Predicting Subscriber Dissatisfaction. *IEEE Transactions on Neural Networks*, 11(3), 690–696.
12. Amin, A., et al. (2019). Customer Churn Prediction in Telecommunication Industry. *Journal of Business Research*, 94, 290–301.
13. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
14. VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media. <https://jakevdp.github.io/PythonDataScienceHandbook/>
15. Coussement, K., & Van den Poel, D. (2008). Churn Prediction in Subscription Services. *Expert Systems with Applications*, 34(1), 313–327.

Appendices

Appendix A How to Run the Project

Follow these steps to run the complete project from scratch on a Windows machine:

- i. Install Python 3.11 from <https://python.org> and MySQL Server 8.0 from <https://dev.mysql.com>
- ii. Install all required libraries in the VS Code terminal

```
pip install flask pandas numpy scikit-learn matplotlib seaborn mysql-connector-python  
joblib
```

- iii. Open MySQL Workbench, create the CUSTOMER_CHURN_PROJECT database and Customers table, then import the IBM Telco CSV.
- iv. Open the project folder in VS Code and update your database credentials in `sqldatabase_connection.py`.
- v. Double-click the Launch Churn App.bat desktop shortcut (or run `python run_project.py` in terminal).
- vi. Open Chrome and navigate to <http://127.0.0.1:5000>

Appendix B Dataset Information

Dataset: IBM Telco Customer Churn | Source: Kaggle
(<https://www.kaggle.com/datasets/blatchar/telco-customer-churn>) | Records: 7,043
customers | Columns: 21 attributes | License: Open Data Commons ODbL | Files used:
WA_Fn-UseC_-Telco-Customer-Churn.csv

Appendix C Glossary of Technical Terms

Term	Definition
Churn	The discontinuation of a service subscription by a customer.
Classification	Supervised ML task predicting a discrete class label.
Confusion Matrix	Table showing TP, FP, TN, and FN for a classifier.
F1-Score	Harmonic mean of precision and recall; balanced performance metric.
Feature Scaling	Normalising numerical features to a common range.
Flask	Lightweight Python web framework for building web applications.
Gradient Boosting	Sequential ensemble ML method correcting prior model errors.

Jinja2	Python templating engine used by Flask for dynamic HTML.
Label Encoding	Converting categorical string values to integer representations.
Logistic Regression	Linear model estimating probability of binary class membership.
MySQL	Relational database management system using SQL.
Pandas	Python library for data manipulation using DataFrame structures.
Precision	Proportion of positive predictions that are actually correct.
Random Forest	Ensemble of decision trees trained on bootstrapped data subsets.
Recall	Proportion of actual positives correctly predicted as positive.
Scikit-learn	Python library providing ML algorithms and evaluation tools.
StandardScaler	Preprocessor standardising features to zero mean and unit variance.

— *End of Major Project Report* —